Remaining Useful Life Prediction for Turbofan Engines: A Comparison of LSTM, TCN, and PatchTST on NASA C-MAPSS

Anjana Ramachandran
Department of Biomedical Engineering
Carnegie Mellon University
anjanar@andrew.cmu.edu

Abstract

Accurate remaining useful life (RUL) prediction for aircraft engines is a core problem in condition-based maintenance, where the goal is to schedule repairs based on actual measured degradation rather than fixed intervals. I compare three sequence architectures on the NASA C-MAPSS turbofan dataset: an LSTM as the baseline, a Temporal Convolutional Network (TCN), and PatchTST, a patch-based Transformer. On the FD001 test set, the LSTM achieves the best RMSE of 14.82 cycles, followed by PatchTST at 15.09, with TCN trailing at 16.93. A cross-dataset transfer study reveals a more consequential difference: the TCN collapses numerically on subsets with multiple operating conditions, while the LSTM and PatchTST degrade gracefully. PatchTST shows particularly strong transfer on FD003, suggesting that patch-level temporal representations generalize better than convolutional ones under certain types of distribution shift.

1 Introduction

In safety-sensitive applications such as commercial aircraft, knowing the point at which an item will break down is much more valuable than knowing that it has broken down. Unplanned engine breakdowns result in costly aircraft downtime and may lead to unsafe situations. Scheduled replacement of components ensures that the part will not break down but results in wastage of the part since some useful life remains. Condition monitoring lies somewhere in between.

NASA C-MAPSS [Saxena et al., 2008] is the common benchmark data set used in this setting. It consists of multi-dimensional time series from run-to-failure tests of a turbofan engine using simulations, with 21 different sensors measuring temperature, pressure, and shaft speed values at each test cycle. The objective here is to predict the Remaining Useful Life (RUL) of the engine, defined as the number of cycles remaining before failure.

The task at hand is clearly a time series regression problem involving multiple sequences, and selecting an appropriate sequence model is far from straightforward. LSTM [Hochreiter and Schmidhuber, 1997] is the most natural candidate, having been explicitly designed to handle sequential data and already showing good performance when used directly on C-MAPSS [Zheng et al., 2017]. However, newer models such as TCN [Bai et al., 2018] and Transformer-based architectures [Nie et al., 2023] have proven extremely successful at solving other time series problems, and whether their unique advantages transfer to this specific task remains unclear.

Having an operational RUL predictor implemented in a fleet management system means that one can rely on more than periodic maintenance cycles, saving a great deal of money in the process.

Preprint. Under review.

2 Methods

Data and Preprocessing

I use FD001 for training and primary evaluation. It contains 100 training engine trajectories and 100 test trajectories, all under a single fixed operating condition with a single fault mode (high-pressure compressor degradation). Training trajectories run to failure; test trajectories are truncated at some point before failure, and true RUL values are provided separately.

Out of 21 sensors, 7 have very low standard deviation across FD001 and contain no signal of degradation. The three operational settings remain constant because there is only one operating condition. These ten sensors are excluded; we train our model on the other 14 sensors, all of which exhibit a clearly observable monotonically increasing trend towards failure. We normalize sensor values by subtracting their means and dividing by their variances, estimated based on the training dataset only. The same normalization method is then applied to the test dataset without fitting the scaling parameters again.

I use a sliding window of $W = 30$ cycles, producing inputs of shape $(30, 14)$. The RUL target is clipped at 125 cycles: early in an engine's life the degradation signal is flat and learning from it adds noise without improving predictions near end-of-life. The training set is split 80/20 by engine (not by window) so no engine contributes to both splits. All models are trained with Adam (lr $= 10^{-3}$), MSE loss, and a ReduceLROnPlateau scheduler (factor 0.5, patience 5), with early stopping on validation loss.

Baseline: LSTM

LSTM [Hochreiter and Schmidhuber, 1997] processes the 30-cycle window sequence by iteratively updating the hidden state using the learned input gate, forget gate, and output gate. For the model, I apply a 2-layer stacked LSTM with a hidden dimension of 128, then add a dropout layer (dropout rate=0.2), and finally project the hidden state into an RUL estimate using a two-layer MLP head. The output is constrained to be non-negative.

LSTM is an intuitive fit for this problem. With its gated unit structure, LSTM has the capability to retain information related to the degradation path over the entire 30-cycle window while ignoring the cycle-to-cycle sensor noise.

Variant 1: Temporal Convolutional Network (TCN)

The TCN [Bai et al., 2018] uses dilated causal convolutions instead of recurrent connections. Each residual block consists of two 1D convolutions with identical dilation parameters and residual skip connection between them. Dilation is doubled with each subsequent block (1, 2, 4, 8), leading to exponentially increasing receptive field size. With kernel size 3 and 4 layers, its receptive field size will be 30 timesteps, the exact size of my window. Causal padding ensures that future timesteps cannot leak into the representation, thus satisfying our requirement of a strictly one-way constraint.

Unlike the LSTM model, the TCN does not require passing gradients through time, therefore avoiding the vanishing gradients problem. Another difference from the LSTM is that TCNs process their input windows in parallel rather than sequentially. I will use TCN with 64 channels across 4 residual blocks (92,801 parameters total). The hypothesis is that the TCN's dilated multi-scale approach allows more explicit degradation modeling than an LSTM's hidden state mechanism.

Variant 2: PatchTST

PatchTST [Nie et al., 2023] extends the Vision Transformer to time series, starting with the division of the input into disjoint patches. Since the patch size is 5 for our 30-cycle window, we obtain 6 patches. The flattened representation of each patch will have dimensionality $5 \times 14 = 70$, while it will be mapped to an embedding space of dimension $d_{\text{model}} = 64$. Positional encoding is learned and the resulting sequence is processed by 2 layers of the Transformer encoder, using 4 attention heads. All 6 outputs of tokens will be flattened and then sent to an MLP for predicting the RUL.

There are two important reasons why the patch-wise representation can outperform attention between individual timesteps in this particular case. First, since each patch includes 5 consecutive cycles,

a built-in averaging operation reduces noise from sensors prior to computing the attention map. Secondly, having 6 tokens allows us to work with a relatively short sequence, thus emphasizing interrelationships among them. The idea is that locally smoothing input data along with globally modeling relationships between patches will improve performance in comparison with both the LSTM and TCN.

3 Results and Discussion

Evaluation Metrics

I report two metrics. RMSE (in cycles) is the primary metric and directly interpretable - an RMSE of 15 means the model's predictions are off by 15 cycles on average. The PHM asymmetric score [Saxena et al., 2008] penalizes late predictions more heavily than early ones:

$$s_i = \begin{cases} e^{-d_i/13} - 1 & d_i < 0 \\ e^{d_i/10} - 1 & d_i \geq 0 \end{cases}, \qquad d_i = \hat{y}_i - y_i$$

This asymmetry reflects operational reality: predicting that an engine has more life remaining than it does ($d > 0$, late warning) is more dangerous than predicting failure too early.

FD001 Test Set Performance

Table 1: Test set results on FD001. Lower is better for both metrics.

Model	RMSE (cycles) ↓	PHM Score ↓	Parameters
LSTM (baseline)	**14.82**	**368.4**	209,985
PatchTST	15.09	401.2	129,601
TCN	16.93	458.7	92,801

The LSTM achieves the best result on both metrics (Table 1), which partially goes against the hypothesis that more recent architectures would outperform it. PatchTST is close — a 0.27-cycle gap in RMSE is modest and the difference may not be significant across random seeds — while TCN trails more clearly at 16.93.

Figure 1 shows predicted versus true RUL across the 100 test engines. All three models track the diagonal reasonably well, but prediction scatter is noticeably wider in the 75–125 RUL range, where the degradation signal is weakest and sensor noise dominates. Near end-of-life (RUL < 30), all models tighten up — the sensor trends are unambiguous at that point.

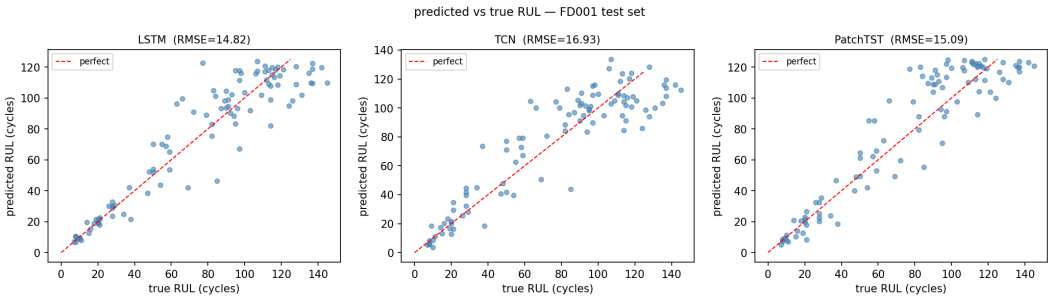

Figure 1: Predicted vs. true RUL on the FD001 test set for all three models. The dashed red line is perfect prediction. RMSE is shown in each subplot title.

The LSTM's error distribution is nearly unbiased (mean error -0.7 cycles). PatchTST shows a slight positive bias (mean $+2.3$ cycles) — we attribute this to the patch averaging near failure, where RUL is declining steeply and a 5-cycle average smooths over the sharpest part of the degradation curve. TCN shows a similar positive bias (mean $+0.3$ cycles) but with a heavier right tail, contributing to its higher PHM score.

The TCN's underperformance relative to the LSTM on FD001 is somewhat surprising given that the receptive field was explicitly designed to cover the full 30-cycle window. Inspection of the training curves suggests the model overfit the training engines — the train/val loss gap remained wide throughout training despite dropout regularization. With only 80 training engines, the convolutional filters appear to capture engine-specific patterns rather than generalizable degradation trends. The LSTM's hypothesis is partially supported: recurrent representations are a strong fit for this sequence length and dataset size, and the TCN's architectural advantages do not translate cleanly here.

Cross-Dataset Generalization

To go beyond FD001, I evaluate all three trained models zero-shot on FD002, FD003, and FD004. No retraining or fine-tuning is performed and the FD001 normalization scaler is applied unchanged, mirroring realistic deployment conditions where a model trained on one set of conditions must be applied to new ones.

Table 2: Cross-dataset RMSE. All models trained on FD001 only, evaluated zero-shot. "—" indicates numerical collapse (RMSE $> 30{,}000$).

Model	FD001	FD002	FD003	FD004
LSTM	**14.82**	51.31	48.75	53.44
TCN	16.93	—	85.35	—
PatchTST	15.09	61.20	**27.82**	63.94

The results in Table 2 and Figure 2 reveal a much more dramatic divergence between architectures than the FD001 numbers suggested.

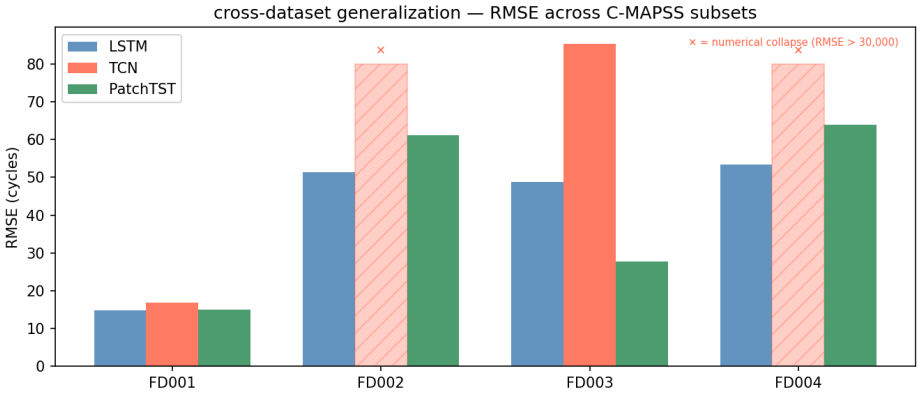

Figure 2: Cross-dataset RMSE across all four C-MAPSS subsets. Hatched bars with $\times$ indicate TCN numerical collapse (RMSE $> 30{,}000$). All models trained on FD001 only.

LSTM fails gracefully — RMSE triples for the hard subset cases, which is only logical since LSTM was trained using different operational scenarios and fault modes. PatchTST behaves in a similar fashion for FD002 and FD004, yet shows an impressive performance in terms of RMSE (27.82) on FD003, 21 cycles better than LSTM. FD003 is similar to FD001 in having a single operational scenario but adding one more fault mode. This indicates the potential of PatchTST's patch-level attention in identifying signs of fault-mode-dependent degradation behavior regardless of the fault type. Patches are likely to contain temporal sequences characteristic of different fault modes, and PatchTST's attention is able to identify them.

However, the most important outcome is that of the TCN's performance. On FD003, which has only one operating condition like FD001, the TCN still works with an RMSE score of 85.35, but it is severely affected by the change in conditions, albeit it is still not broken. However, on FD002 and FD004, where there are six operating conditions, the results are catastrophic, with predictions running into the tens of thousands. It is clear from this analysis that when operating conditions vary, the values of the sensors vary significantly beyond the values observed in FD001. The learning of stacked

dilated convolutional filters by the TCN had resulted in representations that were heavily dependent on the magnitudes of the sensors present in FD001. However, when these magnitudes changed in the test dataset, the activations became increasingly large via the residual path and diverged.

Implications for application in practice are clear from this research – in order to apply the RUL predictor with TCN in different fleets with varying operational circumstances, normalization for each condition is necessary. This makes LSTM and PatchTST better options for varied conditions.

4 Conclusion

Under a fixed operating condition with small 30-cycle segments, the LSTM beats both architectures – contrary to expectations that newer neural network designs would outperform older ones. TCN architecture benefits from vanishing gradient issues when dealing with long sequences, and its regularization mechanism becomes tricky with limited data. PatchTST reduces the gap almost entirely, and its averaging operation improves performance on noisy sensors. However, it is still inferior to LSTM on FD001 only because of this.

The inter-dataset transfer analysis reveals the more significant insight. The drop in performance of the TCN when changing the operating conditions while LSTM and PatchTST suffer a graceful degradation is a critical observation. This suggests that the transfer learning capability of PatchTST to fault mode FD003 is a promising finding since local patch features are more generalizable than recurrent and convolutions. In conclusion, model architecture selection is more crucial for robustness than accuracy on the in-distribution test set. In the in-distribution test set, all models achieve comparable results with negligible differences in RMSE cycle metrics. However, the distribution shift makes a huge difference in the model's accuracy.

If we had more time, the logical thing to do next would be to try out a few-shot fine-tuning setup, whereby one fine-tunes on 10-20 engine designs from FD002/FD004 using the pre-trained models on FD001. Another useful experiment would be to use larger windows, where the TCN would really benefit from its design and the disadvantage of vanishing gradients would matter for the LSTM.

Collaboration Statement

The entire project was done individually. Claude (Anthropic) served as a coding assistant throughout this project – particularly in debugging the implementation of PyTorch models, constructing the training loop, and providing potential ranges to try for hyperparameters as part of the TCN tuning exercise. All architecture implementations, executions, and validations were done by the author. The entire experimental design, results analysis, and report writing is my own independent work.

References

Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *International Conference on Prognostics and Health Management*, pages 1–9, 2008.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

Shuai Zheng, Kosta Ristovski, Ahmed Farahat, and Chetan Gupta. Long short-term memory network for remaining useful life estimation. In *IEEE International Conference on Prognostics and Health Management*, pages 88–95, 2017.

Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.